Pasos 15 y 16 — SPP (System Parameter Permutation): qué se calcula, con qué estadística y qué criterio hay cargado

Fecha: 2026-10-07 · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** sólo los pasos 15 (las dos tareas SPP en SQX) y 16 (su análisis en Python).

Qué es: el documento de consulta del paso 16, con la misma forma que los del paso 8 (`puerta-is-oos-paso-08-especificacion-2026-10-06`) y del paso 10 (`criterios-paso-10-otros-mercados-2026-10-06`). Explica desde cero la estadística que este paso añade, repasa **todos** los números que escribe el estudio, cita el criterio tal como está hoy en `pipeline/autopilot/criteria.yaml` (ninguna regla), y cierra con lo que no cuadra y con las decisiones que te quedan.

De dónde salen los números. Todos los ejemplos son de la corrida `Trade_NIKKEI225_M30_volatilidadL` del 2026-10-04 (NIKKEI225, M30, largas): diez estrategias, 125.708 permutaciones sobre `build` y 125.456 sobre `oos1` . Están **recalculados desde las rejillas exportadas** (`AlgoData/raw/…/SPP_IS` y `SPP_OOS`) con el código del estudio y comparados con los hechos guardados: los 114 hechos por estrategia coinciden (diferencia máxima, en el cuarto decimal). Lo que el estudio no publica como hecho —todo lo de la rejilla `oos1` , la media, la desviación, el rango del real— lo he calculado yo desde las mismas rejillas y lo digo en cada sitio. «Ilustrativo» quiere decir cuenta hecha a mano o simulación para enseñar, no medida.

Ninguna línea de este documento está congelada. El fichero de criterios lo dice en la primera línea de `steps` : provisional hasta que una población sin ventaja haya recorrido la cadena.

---

0. En una página

La pregunta del paso. Si los parámetros de la estrategia hubieran sido otros parecidos, ¿seguiría ganando, o gana sólo con los números exactos que eligió el generador?

Lo que hay cargado hoy: ninguna regla (`rules: []`). El paso corta sobre el databank «SPP OOS», pero sin reglas pasan todas.

Lo que sale en la corrida real: las diez son **Pasa** (calculado en memoria con `pipeline.autopilot.criteria`). El veredicto propio del estudio (`spp.verdict.pass`) también da 1 a las diez.

Las siete cosas que conviene saber antes de leer el resto:

1. **`spp.verdict.area_de_meseta` es el porcentaje de vecinos rentables.** El código la calcula como la fracción de combinaciones distintas con Retorno/Drawdown mayor que cero (`core/surface/plateau.py:24`). El glosario de la ventana la describe como «parte de la rejilla cuyo resultado queda cerca del mejor nivel», que es otra medida (`above_half_max`) que el estudio calcula y no publica. El informe anterior copió el glosario. Es decir: el «70 % de Kaufman» **sí** se puede leer hoy, con ese hecho, sobre la rejilla del `build` .

2. **Todos los hechos del veredicto y del panel 1 son de la rejilla del `build` (IS), no de `oos1`.** El paso se llama «corte sobre SPP OOS» y declara `segment: oos1`, pero el estudio se lanza sobre `SPP_IS` y el extractor de hechos sólo guarda el periodo por defecto del panel, que es «Solo IS». De la rejilla `oos1` no hay ningún hecho propio; sólo entra mezclada con la IS en el panel 2.
3. **El veredicto propio (`pass`, `score`) mide la cola derecha del Retorno/Drawdown, no la robustez.** En siete de las diez estrategias el «mejor punto» de la rejilla es una permutación con menos de 30 operaciones y un drawdown diminuto (el extremo: 6 operaciones, drawdown de 11 USD, Ret/DD = 554,58). Con un suelo de 30 operaciones, los `score` de 17 y 19 bajan a 2,6 y 3,2, y una de las diez (`14.3.479`) pasaría a RUIDO.
4. **El mismo veredicto, por construcción, aprueba un pico y suspende una meseta.** En el ejemplo 3×3 del apartado 2.2, la rejilla con un único punto rentable da `score` 1,31 (SEGUIR) y la rejilla donde las nueve ganan da 0,83 (RUIDO). No es un fallo de programación: la prueba contesta «¿hay algo que sobresalga de la rejilla?», que sirve para decidir si merece la pena fabricar variantes, no para decidir si la estrategia es robusta.
5. **`n_eff` no es un número de pruebas independientes.** Es el número de combinaciones que dieron un backtest distinto (mismo beneficio y mismas operaciones = duplicado). Quita los duplicados exactos; no descuenta que dos vecinos comparten casi todas sus operaciones. Y la discusión «N frente a n efectivo» mueve el listón poco: de 4,33 σ con 11.711 a 3,65 σ con 789. Lo del punto 3 lo mueve diez veces más.
6. **La regla que el Council quería y «no se podía escribir» casi se puede.** «Media de la rejilla menos una desviación típica, negativa» sólo equivale a «el percentil 16 es negativo» en una normal, y aun ahí p10 > 0 es algo más estricto (el p10 de una normal es μ − 1,28σ). El percentil 10 ya es un hecho (`spp.panel1.net_profit.p10`). En la corrida real, las tres formas de aproximarla desde los percentiles y la cuenta exacta fallan a las **mismas tres** estrategias de diez (`15.9.352`, `16.5.352`, `19.2.363`). Con colas largas difieren hasta un factor 2: en C, media − desv = 1.110 y el p16 aproximado = 2.109 (p10 = +69). Que coincidan en las diez es una observación de esta población, no una equivalencia.
7. **Estas diez no son un ensayo justo del paso.** Salieron de un sorteo en el paso 8 y los pasos 10, 12 y 14 no tenían reglas aquel día. Tienen una ventaja para leer este documento: nadie las eligió mirando `oos1`, así que su rejilla `oos1` está razonablemente limpia. En una corrida normal no lo estará (apartado 2.9).

---

1. Qué son los pasos 15 y 16 y cómo deciden

1.1 Dónde están

Familia «¿qué la rompe?» (`studies/breakage/`), junto al MC Retest de los pasos 13-14. El MC Retest cambia el **mundo** (precios, spread, deslizamiento, operaciones saltadas) con la estrategia fija; el SPP cambia la **estrategia** (sus parámetros) con el mundo fijo. Después vienen las variantes (16.5), el WFC (17), el CSCV (18) y la WFM (19), que usan el diseño que este paso deja escrito.

1.2 El paso 15: las dos tareas de SQX

Las configura `python3 -m sqx.projects.spp` (skill `/spp`) desde el bloque `spp:` de `assets/_build.yaml`. Son dos tareas con la misma rejilla y distinta ventana:

tarea	ventana	precisión	qué lee
`SPP IS`	`build`	1	«MCR 8 Stress», lo que dejó el paso 14
`SPP OOS`	`oos1`	1	lo que escribió `SPP IS`

Cómo permuta SQX. Para cada estrategia toma todos sus parámetros numéricos y los cambia **todos a la vez**, al azar, cada uno dentro de ±35 % de su valor original en 18 pasos (unos 4 % por paso). Repite hasta un tope de 15.000 combinaciones y corre un backtest completo por cada una. El backtest original se guarda aparte como «permutación −1».

Lo he comprobado en los datos: en `3.11.426` (10 parámetros) cada parámetro toma entre 15 y 19 niveles, de 0,65 a 1,37 veces el original, y la combinación típica cambia los 10 a la vez (mínimo 7). Dos matices:

- Los parámetros **enteros pequeños** tienen menos niveles: `ExitAfterBars1` = 26 da 11 niveles (16 a 36).
- Los **desplazamientos** (`…Shift1`, `…Shf1`) no se mueven ±35 %: van de 0 a 6 barras, partiendo de 1. Es un cambio mucho mayor que el de los demás.

Por qué 15.000 es una muestra y no la rejilla entera. Con 10 parámetros de 19 niveles hay $19^{10} \approx 6{,}1$ billones de combinaciones posibles. 15.000 es una gota. Por eso las dos tareas no recorren las mismas combinaciones: el estudio midió 6 en común de unas 11.600 en otra estrategia; en las dos que he mirado aquí (`3.11.426` y `15.9.352`) son **cero**.

El tope no se alcanza. Las diez estrategias tienen entre 10.239 y 14.406 filas en `SPP IS`, no 15.000. Por qué SQX se queda corto no está medido.

La aceptación está apagada, a propósito. La tarea trae de fábrica 19 condiciones y cuatro comprobaciones propias del SPP. Una de ellas es exactamente la regla de Kaufman: `ProfitOptPct 70`, «descarta la estrategia si menos del 70 % de sus permutaciones gana». La skill las apaga todas: el paso 15 dibuja un mapa, no filtra. Si SQX filtrase, el paso 16 leería la superficie de una población ya seleccionada sin saberlo.

Costes. Cada tarea cobra los del tramo que corre, según la ficha del activo (`assets/symbols/NIKKEI225.yaml`): en `build`, spread de 304,11 puntos y deslizamiento de 106,44; en `oos1`, 474,91 y 166,22; comisión cero.

Qué costó. Las dos tareas, para diez estrategias, 1.597 segundos (26,6 minutos) en el custodio.

Dos divergencias declaradas (skill `/spp`, `OPEN.md` §38, pendientes de ti): la precisión es 1 y no el 2 de la doctrina (a 1 minuto no termina), y `SPP IS` corre sobre `build`.

1.3 Qué datos lee el paso 16

El estudio `studies/breakage/spp/` lee el export de las dos tareas: una tabla con una fila por permutación (sus parámetros y unas 170 métricas de SQX) y una fila por estrategia con el resumen de SQX.

	`SPP IS`	`SPP OOS`
ventana	`build`, 8 años	`oos1`, 4 años (2020-01 → 2023-12)
filas, diez estrategias	125.708	125.456
filas por estrategia	10.239 a 14.406	10.137 a 14.449

No lee ninguna operación: sólo el resumen de cada backtest. Esa limitación vuelve varias veces en este documento.

1.4 Los tres estados y el corte

Igual que en los demás pasos: **PASA** sigue; **LIMBO** sigue, marcada; **NO PASA** se borra del databank que lee el paso siguiente. Aquí ese databank es «SPP OOS»: de él toma la WFM (paso 19) sus estrategias, y el paso 16.5 las supervivientes para fabricar variantes.

1.5 Cómo se combinan las reglas

`pipeline/autopilot/criteria.py`: el resultado de una estrategia es **el de su peor regla**. Una estrategia sin el hecho que pide una regla queda en Limbo (salvo que la regla diga otra cosa con `missing:`). **Sin reglas, pasan todas** (`criteria.py`, función `outcomes`). Es el caso de este paso.

1.6 Tres avisos

1. **En la corrida real el paso 16 no juzgó.** El resumen del piloto dice «1140 hechos; ninguno: el paso no tiene criterio». Sigue sin tenerlo.
2. **Las diez son un sorteo.** Las mismas del documento del paso 10: «10.000 → 10» en el paso 8, sin mirar el veredicto. Con las reglas del paso 8 de hoy, las diez fallarían allí.
3. **Los hechos de la corrida son 114 por estrategia; hoy saldrían 151.** El mismo día, a las 22:02, el estudio se volvió a ejecutar con las «métricas para criterios» nuevas, que añaden 37 claves `spp.verdict_csv.*`. Están en disco (`AlgoData/reports/…/SPP_IS/2026-10-04/spp/verdict.csv`) y no en el `hechos.parquet` de la corrida. Las explico en el apartado 3.6.

1.7 Las estrategias que nos acompañan

Las tres del documento del paso 10, con su letra, y una cuarta que hace falta aquí porque es la que más se parece a «una elección con suerte».

	C	E	F	G
nombre (`Strategy …`)	`14.3.479`	`3.11.426`	`15.9.352`	`4.13.338`
parámetros permutados	17	10	15	13
operaciones IS / OOS1	189 / 98	135 / 92	224 / 167	198 / 126
neto IS (USD)	13.239	22.508	2.944	53.725
neto OOS1 (USD)	4.690	19.670	5.626	22.721

No hay una de cada veredicto porque no existe: las diez pasan. Para lo que este paso enseña, **F es el caso malo** (gana en el `build`, pero sólo gana el 23 % de sus vecinos), **E y C los normales**, y **G el caso «demasiado arriba»** (su backtest real supera al 94 % de sus vecinos).

---

2. Fundamentos de este paso

2.0 Lo que ya sabes de los pasos 8 y 10, en ocho líneas

```
error estándar = desv / √n
t = media / (desv / √n)
p = 1 − Φ(t)                        Φ(1,65) = 0,95 · Φ(2,33) = 0,99
n efectivo = n × (1 − ρ) / (1 + ρ)          ρ = correlación entre observaciones vecinas
máximo esperado de N tiradas de ruido ≈ σ × √(2 × ln N)
Šidák: p de familia = 1 − (1 − p)^N         Bonferroni: p de familia ≈ N × p
```

El **p-valor** es la probabilidad de que algo sin ventaja llegue ahí por suerte. El **máximo de N pruebas**: si pruebas muchas cosas, la mejor sale bien aunque ninguna valga; por eso se compara con lo que daría la mejor de N tiradas de ruido. El **n efectivo**: observaciones que se parecen cuentan como menos observaciones. Todo está enseñado con ejemplos en el documento del paso 8 (apartado 2) y en el glosario de `criterios-pasos-6-a-19-explicados-2026-10-06`.

Lo que sigue es lo **nuevo** de este paso.

2.1 Qué es la permutación de parámetros y qué contesta

El problema. El generador de SQX probó miles de estrategias y, para cada una, eligió unos parámetros concretos: periodo 20, desviación 2,5, salida a las 26 barras. El backtest que ves es el de **esa** combinación. Pero el 20 no es sagrado: si la idea funciona, también debería funcionar con 18 o con 23. Si sólo funciona con 20, lo que encontró el generador no es una idea: es una coincidencia entre ese número y ese histórico.

La idea de Dave Walton (el artículo que da nombre a la técnica, premio Wagner de la NAAIM en 2014; no está en el paquete de evidencia del Council, lo cito de memoria por ser el origen del nombre): en vez de mirar el único backtest de la combinación elegida, corre **todas las combinaciones parecidas** y mira la **distribución** de resultados.

- La **mediana** de esa distribución es una estimación menos sesgada de lo que el sistema da. La razón: al elegir la mejor combinación eliges, además del mérito, la suerte que tuvo esa combinación en ese histórico. La mediana no arrastra la suerte de haber elegido la mejor combinación, pero sí la del periodo, que comparten todos los vecinos. Y depende de la anchura de la caja que explora SQX (±35 %, centrada en un óptimo ya ajustado): con otra caja, otro número.
- Las **colas** acotan el riesgo: el percentil 5 es «lo que habría pasado si hubieras tenido mala mano al escoger parámetros».
- El **backtest real** se sitúa dentro de esa distribución: si está cerca de la mediana, es un vecino cualquiera; si está en la cola alta, lo elegido fue un extremo.

Qué contesta que el backtest único no contesta. El backtest único da un número. La permutación da tres cosas más: cuánto de ese número es elección afortunada (real menos mediana), cuánta parte del vecindario gana dinero, y cómo de ancho es el abanico (cuánto depende el resultado de acertar con los parámetros).

Qué no contesta. Todos los vecinos se prueban sobre **el mismo histórico**. Si ese histórico fue favorable a la idea por casualidad, lo fue para todos los vecinos a la vez. La permutación mide la sensibilidad a los parámetros; no mide la suerte del periodo. Es el núcleo de los apartados 2.6 y 2.8.

2.2 Un ejemplo con una rejilla de 3×3

Dos parámetros, tres valores cada uno: periodo (14, 20, 26) y barras hasta la salida (8, 12, 16). El generador eligió **periodo 20, salida 12**, y su backtest da **9** (miles de USD). Mira dos mundos posibles con ese mismo backtest.

Rejilla A — meseta

periodo \ salida	**8**	**12**	**16**
14	6	8	7
20	7	**9**	8
26	5	7	6

Rejilla B — pico

periodo \ salida	**8**	**12**	**16**
14	−3	−1	−2
20	−1	**9**	−2
26	−4	−2	−3

Las cuentas, a mano:

```
A ordenada:  5  6  6  7  7  7  8  8  9          B ordenada:  −4 −3 −3 −2 −2 −2 −1 −1  9
suma = 63        media = 63 / 9 = 7,0           suma = −9       media = −9 / 9 = −1,0
mediana = 7 (el 5.º)                            mediana = −2 (el 5.º)
desviaciones²: 4 1 1 0 0 0 1 1 4 = 12           desviaciones²: 9 4 4 1 1 1 0 0 100 = 120
desv = √(12/9) = 1,155                          desv = √(120/9) = 3,651
media − desv = 5,85                             media − desv = −4,65
rentables = 9 de 9 = 100 %                      rentables = 1 de 9 = 11 %  (0 de 8 vecinos)
real − mediana = 9 − 7 = 2                      real − mediana = 9 − (−2) = 11
```

	A (meseta)	**B (pico)**
backtest elegido	9	9
mediana (Walton)	7	−2
media − 1 desv. (Kaufman)	+5,85	−4,65
% rentables (Kaufman)	100 %	11 %

Con el backtest único, A y B son la misma estrategia. Con la rejilla, A es una idea que da unos 7 con casi cualquier parámetro, y B es un número que sólo existe en una casilla. Eso es una **solución pico**: el rendimiento cae en todas las direcciones al moverte de la casilla elegida (aquí, las cuatro vecinas directas valen −1, −1, −2, −2).

Guarda este ejemplo: en el apartado 2.5 se le aplica el veredicto del estudio y sale al revés de lo que esperas.

2.3 Percentiles y la banda

Percentil q: el valor por debajo del cual queda el q % de los vecinos. El estudio usa el método estándar de `numpy` (`core/study/blocks.py:115`): se ordenan los N valores de menor a mayor, se numeran de 0 a N−1, y el percentil q es el valor en la posición

```
posición = (q / 100) × (N − 1)
```

interpolando en línea recta si la posición no es entera.

Ejemplo a mano con la rejilla A (N = 9, posiciones 0 a 8): p25 → posición 0,25 × 8 = 2 → el tercer valor, **6**. p50 → posición 4 → **7**. p75 → posición 6 → **8**. p10 → posición 0,8 → entre el 5 y el 6, a 0,8 del camino: 5 + 0,8 × 1 = **5,8**.

El estudio publica nueve: p1, p5, p10, p25, p50, p75, p90, p95 y p99. **No hay p16** (importa en 2.7).

Rango intercuartílico (IQR) = p75 − p25: la anchura de la mitad central. En A, 8 − 6 = 2. Se usa como «unidad de anchura» que no se deja arrastrar por los valores extremos, a diferencia de la desviación típica.

La banda de la ventana. La ventana pinta mediana ± 30 % de la propia mediana (`config.yaml: panel1.band_share`) y dice si el real cae dentro o fuera. Es el criterio heredado de SQX. Tiene un defecto que el propio código anota (`criteria.py`, ayuda de la columna): premia la poca dispersión y estalla si la mediana está cerca de cero. No es un hecho y ninguna regla lo lee.

2.4 Real frente a mediana: el rango del real como p-valor de sobreoptimización

La pregunta. ¿En qué puesto queda el backtest real entre sus propios vecinos?

```
rango (%) = 100 × (n.º de vecinos con valor MENOR que el real) / N
p de «elección con suerte» = 1 − rango / 100
```

Cómo leerlo. Supón que los parámetros no importan nada dentro del vecindario (todos los vecinos son igual de buenos y sólo los separa el ruido). Entonces el real sería un vecino más y su rango sería un número al azar entre 0 y 100. Un rango de 95 quiere decir: «sólo el 5 % de los vecinos hace tanto o más que el elegido». O los parámetros elegidos son de verdad mejores, o el generador se quedó con el vecino que más suerte tuvo. Con un solo histórico no se pueden separar las dos cosas; lo que sí dice el rango es **cuánto del resultado se pierde si la suerte no se repite**: la diferencia entre el real y la mediana.

Ejemplo real, G en el `build`, Retorno/Drawdown. 13.968 vecinos. El real vale 8,65; la mediana de la rejilla, 4,35. De los 13.968, el real supera al 93,9 %.

```
rango = 93,9 %          p = 1 − 0,939 = 0,061
real − mediana = 8,65 − 4,35 = 4,30      (el real es 1,99 veces la mediana)
```

Lectura: el 8,65 es el dato de una combinación que quedó entre el 6 % mejor de su vecindario. Lo que cabe esperar de «esta idea con parámetros razonables» es 4,35, no 8,65.

El rango se puede aproximar con los hechos que ya existen. El real (8,65) cae entre p90 (7,80) y p95 (8,95):

```
rango ≈ 90 + 5 × (8,65 − 7,80) / (8,95 − 7,80) = 90 + 5 × 0,739 = 93,7     (exacto: 93,9)
```

Dos cuidados.

- **Es una prueba entre muchas.** Con diez estrategias, que alguna llegue al 93,9 % por puro azar tiene probabilidad $1 - 0{,}939^{10} = 47$ %. Un rango alto en una de diez no señala nada; lo que informa es el conjunto.
- **Dentro de muestra se espera un rango alto.** El generador eligió esos parámetros *porque* salían bien en el `build`. Lo anómalo sería un rango de 50. Por eso el rango IS mide cuánto apretó el generador; el rango en `oos1` es la lectura limpia, siempre que `oos1` no se haya usado para elegir (apartado 2.9).

Lo que sale en las diez (calculado por mí desde las rejillas; **no es un hecho**):

rango del real	IS: mediana (mín–máx)	OOS1: mediana (mín–máx)
beneficio neto	68 (28–90)	52 (32–80)
profit factor	60 (30–87)	63 (30–93)
Retorno/Drawdown	66 (30–94)	59 (31–85)
Sharpe	66 (33–92)	56 (30–91)

En el `build`, la media del rango en Ret/DD es 62,3 con desviación 20,9: error típico 20,9/√10 = 6,6, a 1,9 errores de 50. El generador empujó el real algo por encima de su vecindario, pero no lo dejó en la cola: ninguna de las diez llega al 95 % en beneficio, Ret/DD o Sharpe. En `oos1` la media es 57,5 (error 5,4; a 1,4 errores de 50): con diez estrategias y un error típico de 5 a 7 puntos no se puede afirmar ni que el puesto se conserve ni que no: F y G bajan, E sube de 64 a 76, y parte de la bajada es simple regresión a la media.

Las cuatro que seguimos, en Retorno/Drawdown:

	rango IS	rango OOS1
C `14.3.479`	44	43
E `3.11.426`	64	76
F `15.9.352`	79	41
G `4.13.338`	94	61

Con diez estrategias y un error típico de 5 a 7 puntos no se puede afirmar ni que el puesto se conserve ni que no: F y G bajan, E sube de 64 a 76, y parte de la bajada es simple regresión a la media.

2.5 El veredicto propio del estudio: la prueba de ruido

Hechos: `spp.verdict.score`, `spp.verdict.pass`.

La pregunta que hace (`verdict/noise.py:11`): bajo la hipótesis de que **ningún parámetro hace nada**, los resultados de la rejilla serían ruido alrededor de su media. El mejor de N tiradas de ruido con desviación σ queda, aproximadamente, a $\sigma \times \sqrt{(2 \ln N)}$ por encima de la media. Si el mejor punto de la rejilla no supera

eso, la rejilla «no ha encontrado nada» y no merece la pena fabricar variantes.

La fórmula exacta. Se calcula sobre el **Retorno/Drawdown** (`config.yaml: read.metric`) de la rejilla del export elegido (en la corrida, `SPP_IS`), con el original dentro y sin duplicados:

```
valores  = Ret/DD de cada combinación con resultado distinto
n_eff    = cuántas son
σ        = desviación típica de esos valores
exceso   = máximo − media
listón   = σ × √(2 × ln n_eff)
score    = exceso / listón
pass     = 1 si exceso > listón × margen          margen = 1,0 (config.yaml: verdict.margin)
```

No hay suelo de operaciones (decisión tuya del 2026-10-04, anotada en el código).

Ejemplo real, C. n_eff = 12.468; σ = 2,5643; máximo = 19,44; media = 3,0831.

```
ln 12.468 = 9,431          √(2 × 9,431) = √18,862 = 4,343
listón = 2,5643 × 4,343 = 11,137
exceso = 19,44 − 3,0831 = 16,357
score  = 16,357 / 11,137 = 1,4687       → hecho guardado: 1,4687  ✓   → pass = 1 (SEGUIR)
```

Ahora aplícalo al ejemplo 3×3 (N = 9, √(2 ln 9) = 2,096):

```
A (meseta):  exceso = 9 − 7 = 2       listón = 1,155 × 2,096 = 2,42     score = 0,83  → RUIDO
B (pico):    exceso = 9 − (−1) = 10   listón = 3,651 × 2,096 = 7,65     score = 1,31  → SEGUIR
```

La prueba aprueba el pico y suspende la meseta. No es un error: contesta lo que pregunta. «¿Sobresale algo de esta rejilla?» En B sí (un punto, muchísimo); en A no (todo es parecido). El README del estudio lo sabe y lo dice: `score` «se lee junto con» el área y el pico, «nunca en su lugar». La pregunta para la que se escribió es «¿merece esta familia las 5.000 variantes?», no «¿es robusta esta estrategia?». **Leer `spp.verdict.pass` como «pasa el SPP» es leer lo contrario de lo que mide.**

Qué es el «mejor punto» en los datos reales. La permutación con más Ret/DD de cada rejilla IS:

estrategia	Ret/DD máx.	operaciones	drawdown (USD)	`score`
`13.3.390`	554,58	6	11	17,11
`3.9.284`	624,86	7	8	19,31
`18.5.355`	122,94	18	147	10,41
E `3.11.426`	69,57	26	276	5,64
G `4.13.338`	18,35	200	3.067	1,28

Un Ret/DD de 554 es 6.211 USD ganados en seis operaciones con una caída máxima de 11 USD: una combinación de parámetros que casi nunca entra y no ha tenido tiempo de perder. En **siete de las diez** el máximo tiene menos de 30 operaciones (6, 7, 16, 18, 20, 26 y 29). El p99 del Ret/DD de esas mismas rejillas

está entre 1,75 y 14,4: el máximo es un punto suelto muy lejos del resto.

Con un suelo de 30 operaciones (calculado por mí; no es lo que hace el código): los `score` pasan de 1,28–19,31 a 0,85–3,96. Los de 17,11 y 19,31 quedan en 2,59 y 3,18. C queda en 0,85: **RUIDO**. Nueve de diez seguirían pasando.

2.6 `n_eff`, N independientes y las dos críticas

Hechos: `spp.summary.n_eff`, `spp.verdict.tuplas_distintas_n_eff` (son el mismo número).

Qué es `n_eff` aquí (`core/surface/dedupe.py`, función `distinct`): el número de filas de la rejilla tras quitar las que repiten **a la vez** beneficio neto y número de operaciones. Dos combinaciones con el mismo beneficio y las mismas operaciones corrieron el mismo backtest; pasa cuando un parámetro es inerte (cambiarlo no cambia ninguna operación).

Ejemplo real, C: 14.406 filas, 12.468 distintas. F: 14.060 filas, 10.357 distintas (tiene 8 parámetros inertes de 15: `spp.summary.frozen` = 8).

Por qué se usa en vez del número de filas. El listón σ × √(2 ln N) sube con N. Contar duplicados como pruebas nuevas subiría el listón contra una rejilla más pequeña de lo que parece.

Qué NO es. No es un «número de pruebas independientes» en el sentido del paso 8 o del 10. Dos vecinos que difieren en un 4 % de un periodo no son duplicados, pero comparten casi todas sus operaciones: sus resultados están muy correlacionados. `n_eff` no descuenta eso. Para medirlo haría falta la lista de operaciones de cada permutación, que el export de SQX no trae. **La correlación entre vecinos no está medida.**

Cuánto podría valer. Con la fórmula del paso 8, n × (1 − ρ)/(1 + ρ), y 15.000 pruebas:

correlación ρ	pruebas independientes	listón √(2 ln N)
0 (lo que supone el código)	15.000	4,39 σ
0,5	5.000	4,13 σ
0,9	789	3,65 σ
0,99	75	2,94 σ

Esa fórmula es la de una serie en el tiempo; en una superficie de diez parámetros los n efectivos de la tabla (5.000, 789, 75) son hipotéticos, no medidos. Y √(2 ln N) es el máximo esperado del ruido, no un umbral al 5 %. Sirve para ver el orden de magnitud: **el listón se mueve poco**, porque depende del logaritmo de N. Dividir N por 19 (de 15.000 a 789) baja el listón un 17 %.

Las dos aritméticas, Bonferroni y Šidák. Un punto a 4,4 σ tiene, él solo, p = 0,0000054 (una cola). La probabilidad de que **el mejor de N** llegue ahí por suerte:

N	Šidák: 1 − (1 − p)^N	Bonferroni: N × p
15.000	7,8 %	8,1 %
11.711 (el `n_eff` de un caso real)	6,1 %	6,3 %
789	0,43 %	0,43 %
50	0,03 %	0,03 %

Y al revés, el listón que deja el 5 % de falsos positivos en la familia entera: 4,50 σ con N = 15.000; 4,45 σ con 11.711; 3,83 σ con 789; 3,09 σ con 50.

Crítica 1 — el Council: el listón es demasiado estricto (ronda 1: «la prueba de ruido sobre 15.000 pruebas correlacionadas, 4,4 sigma, es sobre-estricta»). Si las pruebas independientes de verdad son 789 y no 11.711, el listón justo sería 3,65 σ y no 4,33 σ: el código exige un 19 % de más. Lo he comprobado con una simulación ilustrativa (400 rejillas de 11.700 puntos de ruido normal):

rejilla de puro ruido	`score` mediano	pasa (`score` > 1)
puntos independientes	0,89	7,5 %
vecinos correlacionados 0,99	0,74	1,2 %
vecinos correlacionados 0,999	0,59	0 %

Con ruido normal, cuanto más correlacionados los vecinos, **menos** pasa. El Council tiene razón en la dirección.

Crítica 2 — el revisor del informe anterior: el `score` sobrestima («supone N tiradas independientes cuando los vecinos están correlacionados, así que sobrestima: que las diez pasen no es una señal fiable»). Tomada al pie de la letra, la razón está al revés: la correlación baja el `score`, no lo sube (tabla de arriba). Pero la conclusión es cierta por otro motivo, que es el que de verdad manda:

El modelo «ruido normal alrededor de la media» no describe un Retorno/Drawdown. Ret/DD es un cociente, beneficio entre caída. Cuando una combinación opera poco, la caída es diminuta y el cociente se dispara. La misma simulación con un cociente sin ventaja alguna (numerador de media cero, denominador que a veces se acerca a cero; ilustrativo):

rejilla sin ventaja	`score` mediano	pasa
cociente tipo Ret/DD	12,7	96 %
valores uniformes (acotados)	0,40	0 %

Con el cociente simulado, que es arbitrario, una rejilla **sin ventaja ninguna** pasa el 96 %; el porcentaje depende del cociente elegido y sólo enseña que una cola larga puede hacer pasar ruido (con una métrica acotada no pasa nunca).

Balance. Las dos críticas son ciertas, cada una sobre un efecto distinto, y no se compensan: la correlación cambia el listón entre un 17 y un 33 %; la cola del cociente cambia el `score` por diez (lo visto en 2.5: 17,1 → 2,6 con un suelo de 30 operaciones). En la corrida real pasan las diez; con el listón más bajo que pediría el Council pasarían igualmente las diez. **`pass` no separa a nadie en esta población, en ninguna de las dos versiones.**

2.7 El porcentaje de vecinos rentables, Kaufman y por qué informa poco

Hecho: `spp.verdict.area_de_meseta` (es esto, pese al nombre: apartado 5.1).

La regla de Kaufman (paquete de evidencia del Council, apartado «Parameter robustness»):

- robustez = porcentaje de pruebas de parámetros rentables; se acepta con **70 % o más**, sobre un rango fijado de antemano (en su estudio lo pasaron 11 de 17 mercados);
- lo que se debe esperar es la **media** de la rejilla, o la **media menos una desviación típica**, nunca la mejor casilla;
- hay **solución pico** si el rendimiento cae en todas las direcciones;
- un filtro sólo cuenta si sube el porcentaje de combinaciones rentables.

En el 3×3: A tiene 100 % y media − desv = +5,85 → pasa las dos. B tiene 11 % y −4,65 → falla las dos, y además es un pico.

Por qué es bimodal. Todos los vecinos operan sobre el mismo histórico. Escribe el resultado de cada vecino como una parte **común** (la suerte del periodo, igual para todos) más una parte **propia**:

```
resultado del vecino i = √ρ × c + √(1 − ρ) × eᵢ        c, eᵢ ~ normales de media 0 y desviación 1
fracción de vecinos rentables, dado c  =  Φ( c × √(ρ / (1 − ρ)) )
```

ρ es la correlación entre vecinos; c es la suerte común. He puesto **ventaja cero**: la media de todo es 0.

Con ρ = 0,9, √(0,9/0,1) = 3, así que la fracción es Φ(3c):

suerte común c	−1	−0,5	−0,2	0	0,2	0,5	1
vecinos rentables	0,1 %	6,7 %	27 %	50 %	73 %	93 %	99,9 %

Un cambio pequeño en la suerte del periodo (de −0,5 a +0,5 desviaciones) lleva el porcentaje del 7 % al 93 %. Como c es un número al azar, el porcentaje casi nunca cae en medio:

```
P(más del 90 % rentables)   = P(c >  1,2816/3 = 0,427) = 33,5 %
P(menos del 10 % rentables) = 33,5 %
P(entre 10 % y 90 %)        = 33,1 %
P(70 % o más)               = P(c > 0,5244/3 = 0,175)  = 43,1 %
```

Eso es «bimodal»: dos montones, uno cerca de 0 % y otro cerca de 100 %, y poca cosa en medio. **Una familia sin ventaja pasa el 70 % de Kaufman el 43 % de las veces.** Y las que llegan al paso 16 no son familias al azar: se eligieron porque ganaban en el `build`, es decir, con c positivo. Condicionado a c > 0, la probabilidad es 0,431 / 0,5 = **86 %**: con ρ = 0,9 supuesto (no medido), el modelo da un 86 %; con ρ menor baja y con vecinos no gaussianos puede cambiar. Es una ilustración, no una tasa de falsos positivos.

El modelo es ilustrativo y ρ = 0,9 es un supuesto (no está medido). La conclusión no depende del número exacto: el porcentaje de vecinos rentables **mide sobre todo cómo le fue al periodo**, que es algo que ya sabías por el backtest. Aporta poca información nueva.

En los datos reales. Porcentaje de vecinos con Ret/DD positivo, las diez, ordenado:

```
build (el hecho):   23   68   83   89   96   96   97   97   98   99,6
oos1 (calculado):   44   69   75   75   86   88   91   94   95   98
```

En el `build`, seis de diez por encima del 95 %; cuatro por debajo (F con 23, `19.2.363` con 68, y otras dos con 83 y 89). La forma es la del modelo: amontonadas arriba.

El aviso de Katz (misma fuente): un caso con **todas** las combinaciones de parámetros rentables dentro de muestra y **ninguna** fuera. Una meseta dentro de muestra no es robustez: es que el periodo fue bueno para toda la familia.

Dos ejemplos de nuestra propia corrida, en las dos direcciones:

estrategia	rentables en `build`	rentables en `oos1`
`13.3.390`	97 %	44 %
F `15.9.352`	23 %	75 %

La primera es el caso de Katz en pequeño: meseta perfecta en el `build`, y fuera pierde más de la mitad del vecindario (la mediana de su rejilla `oos1` es −303 USD). La segunda es lo contrario: la regla del 70 % la habría matado y fuera gana el 75 % de sus vecinos.

¿Y en conjunto? Entre las diez, la correlación de rangos entre el porcentaje del `build` y el de `oos1` es +0,59, con un intervalo del 95 % de −0,07 a +0,89. Con diez estrategias de una misma plantilla no se puede decir si la meseta del `build` predice la de fuera. **No medido** en una población grande.

Lo que dijo el Council (ronda 1): «el % de vecinos rentables es bimodal, poca información; Katz: meseta ≠ robustez. Sólo descarte: solución pico, o media de la rejilla menos 1 desv. < 0. El ≥ 70 % de Kaufman como convención de Pasa, pero nunca como única evidencia.» Dos de los miembros se pusieron «del lado de las mediciones contra el 70 % de Kaufman».

2.8 Qué aspecto numérico tiene una solución pico

Kaufman la define por la forma: el rendimiento cae **en todas las direcciones** desde el punto elegido. En números, sobre una rejilla, se reconoce por cuatro señales juntas:

señal	meseta (A)	pico (B)
rango del real	89 % (8 de 9)	89 % (8 de 9)
(real − mediana) / IQR	2 / 2 = 1,0	11 / 2 = 5,5
% rentables	100 %	11 %
media − desv	positiva	negativa

La segunda fila es la clave: la distancia del real a la mediana **medida en anchuras del vecindario**. En A el elegido está a una anchura de la mediana; en B, a cinco y media. El rango es igual en las dos, porque el real es el máximo de ambas: no distingue meseta de pico. Lo que las separa es la distancia a la mediana en IQR (1,0 frente a 5,5) y el porcentaje de vecinos rentables (100 % frente a 11 %).

En los datos reales no hay ninguna. La distancia del real a la mediana en IQR (Ret/DD, `build`) va de −0,35 a +1,32 en las diez; la mayor es G (+1,32). Ninguna está fuera del rango en que cae el 90 % de sus propios vecinos.

El estudio tiene además una señal por parámetro, que no es un hecho: en el `design_brief` de cada estrategia marca `spike` cuando la curva de **un** parámetro tiene una meseta de un solo nivel. Entre las diez hay de 0 a 4 parámetros así por estrategia (de 6 a 14 vivos), y el original coincide con el mejor nivel en 0, 1 o 2 parámetros, nunca en todos. Con todos los parámetros moviéndose a la vez, «cae en todas las direcciones» sólo se puede comprobar parámetro a parámetro; la vecindad inmediata del original (todos los parámetros a un paso) está prácticamente vacía en una muestra de 12.000 puntos de un espacio de billones.

2.9 Por qué «SPP OOS» deja de ser fuera de muestra, y por qué el Council no puso regla

Qué significa «SPP OOS». Es la misma permutación corrida sobre `oos1`, el tramo que el generador no vio. La idea es buena: si los vecinos también ganan en un tramo que nadie usó para elegir parámetros, la familia vale.

El problema. En una corrida normal, a este paso sólo llegan las estrategias que **ya pasaron el paso 8**, y el paso 8 las eligió mirando `oos1`. La estrategia que llega aquí es la que tuvo un buen `oos1`. Sus vecinos comparten casi todas sus operaciones de `oos1`. Así que su rejilla `oos1` sale bien **porque ya se la escogió por eso**. No es una muestra nueva: son las mismas operaciones leídas otra vez con los parámetros un poco movidos.

Es la frase del Council (ronda 1): «sólo el paso 8 tiene una tasa de falsos positivos medida; todo lo que viene después relee los mismos pocos centenares de operaciones». Y por eso este paso quedó sin regla de Pasa: **sobrevivir aquí no es evidencia**. Lo más que puede hacer es descartar lo que esté claramente mal (sólo descarte).

A eso se suman tres razones concretas:

1. **Las dos condiciones de descarte que quería** —solución pico, y media de la rejilla menos una desviación negativa— **no están expuestas como hecho**. Es literalmente lo que dice el comentario de `criteria.yaml`.
2. **Ninguna tasa medida.** No se sabe cuántas estrategias sin ventaja pasarían cualquier regla de este paso. Un miembro propuso la prueba: pasar «monos seleccionados» por los pasos 14 y 16; si más del 90 % de los nulos pasa, la prueba es vacía.
3. **Las métricas de decaimiento IS → OOS1 que el estudio sí calcula** (apartado 3.6) no distinguen una familia sin ventaja de una real con un `oos1` de 4 años (`knowhow/research/spp-decay-metrics-zero-edge-null.md`): una familia sin ventaja tiene su rejilla `oos1` en Sharpe 0 ± 1/√4 = ±0,5, y eso cubre «no decae» para casi cualquier mediana IS.

El estado en disco hoy:

```
"16":
  cut: SPP OOS
  segment: oos1
  # Council: FAIL-only on a peak solution / grid mean - 1 sd < 0; neither is exposed as a fact
yet, so no rule.
  rules: []
```

---

3. Lo que calcula el estudio, pieza a pieza

El `hechos.parquet` del paso 16 de la corrida tiene **1.140 filas: 114 claves × 10 estrategias**, todas del estudio `spp`. Agrupadas:

grupo	claves	de qué rejilla
`spp.summary.*`	3	la del export elegido (`SPP_IS`)
`spp.verdict.*`	5	la misma
`spp.panel1.<métrica>.*`	6 × 11 = 66	`build` , sin el original
`spp.panel2.<métrica>.*`	4 × 10 = 40	`build` y `oos1` **mezcladas**
`spp.verdict_csv.*`	37 (hoy; no en la corrida)	las dos

Para cada pieza: qué mide, cómo se calcula, ejemplo y cómo leerlo. Los rangos «p5 / mediana / p95» son sobre **diez** estrategias: orientan, no calibran.

3.1 El resumen: `spp.summary.*`

`spp.summary.n_eff` — combinaciones con resultado distinto. Explicado en 2.6. (`verdict/noise.py:33` .) Rango: 10.255 / 11.470 / 12.701. C: 12.468. E: 10.493. F: 10.357. G: 11.901.

`spp.summary.live` y **`spp.summary.frozen`** — cuántos parámetros **cambian** algún backtest (vivos) y cuántos **no cambian ninguno** (congelados).

- *Cómo se calcula* (`run.py` , función `brief`): por la **prueba de duplicados**. Para un parámetro, se agrupan las filas que coinciden en todos los demás parámetros; si en **todos** los grupos el resultado es idéntico, el parámetro es inerte y se congela en el diseño de variantes. No se decide por «cuánta varianza explica», porque un parámetro inerte puntúa por encima de cero en esa medida (el código lo documenta con un caso medido).
- *Ejemplo real:* C tiene 17 parámetros permutados: 14 vivos y 3 congelados. F: 15 permutados, 7 vivos y **8 congelados**. E: 10, 6 y 4. G: 13 vivos, 0 congelados.
- *Cómo leerlo:* no es bueno ni malo. Un congelado es un parámetro que la estrategia tiene pero no usa (o cuyo rango de ±35 % nunca cruza un valor que cambie una operación). Muchos congelados quieren decir que la rejilla efectiva es más pequeña de lo que parece. Rango de vivos: 6 / 8 / 13,6.

3.2 El veredicto: `spp.verdict.*`

Todo sobre el Retorno/Drawdown de la rejilla `build` , sin duplicados y **con el original dentro**.

`spp.verdict.score` y **`spp.verdict.pass`** — la prueba de ruido. Fórmula y ejemplos en 2.5.

	n_eff	σ	máx.	media	listón	`score`
C	12.468	2,564	19,44	3,083	11,137	1,469
E	10.493	2,738	69,57	3,174	11,782	5,636
F	10.357	0,563	3,19	−0,240	2,419	1,418
G	11.901	2,491	18,35	4,503	10,792	1,283

Los cuatro `score` coinciden con el hecho guardado. *Cómo leerlo:* por encima de 1, «algo sobresale de la rejilla». No dice que la estrategia sea buena ni robusta; F, la peor de las diez por vecindario, pasa. Rango: 1,34 / 4,12 / 18,3. `pass` = 1 en las diez.

`spp.verdict.area_de_meseta` — el porcentaje de vecinos rentables.

- *Qué mide:* qué parte de las combinaciones distintas tiene Ret/DD mayor que cero.
- *Cómo se calcula* (`core/surface/plateau.py:24`): `(valores > 0).mean()`, sobre las combinaciones distintas.
- *Ejemplo real, C:* de 12.468 combinaciones distintas, el 88,99 % tiene Ret/DD > 0 → 0,8899 ✓. F: 0,2283. E: 0,9773. G: 0,9960. El ejemplo del encargo, 0,9629, es `10.11.484`.
- *Cómo leerlo:* cerca de 1, casi todo el vecindario gana en el `build`. Es el número de Kaufman; por debajo de 0,70 falla su regla. Con las limitaciones de 2.7. Rango: 0,43 / 0,96 / 0,99.
- *Un matiz:* al quitar duplicados, el porcentaje cambia. En `13.3.390` hay 333 combinaciones que no operan nunca (beneficio 0, cero operaciones); son un solo «resultado distinto», así que sobre filas el porcentaje es 94,8 % y sobre distintas 97,3 %. En F, al revés: 29,7 % sobre filas, 22,8 % sobre distintas.

`spp.verdict.max_mediana_iqr` — cuánto sobresale el mejor punto.

- *Cómo se calcula* (`verdict/noise.py:60`): `(máximo − mediana) / IQR`, sobre las combinaciones distintas.
- *Ejemplo real, C:* máximo 19,44; mediana 2,76; el hecho vale 4,1883, así que el IQR es (19,44 − 2,76)/4,1883 = 3,98. Es decir: el mejor punto está a 4,2 anchuras del centro. El ejemplo del encargo, 7,12, es `10.11.484`. En el 3×3: A da 1,0 y B da 5,5.
- *Cómo leerlo:* alto = el máximo es un punto aislado. Pero cuidado: mide el **mejor punto de la rejilla**, no el **backtest elegido**. Un valor de 249 (`13.3.390`) dice que hay una permutación de 6 operaciones con un Ret/DD absurdo; no dice nada de la estrategia. Para saber si el *elegido* es un pico hay que mirar la distancia del real a la mediana (apartados 2.8 y 3.6). Rango: 4,2 / 15,7 / 224.

`spp.verdict.tuplas_distintas_n_eff` — el mismo número que `spp.summary.n_eff`, publicado otra vez como parte del veredicto.

Lo que el veredicto calcula y no publica: `above_half_max`, la fracción de la rejilla que queda por encima de la mitad del máximo (entre 0,0001 y 0,044 en las diez; es a lo que se refiere el glosario cuando describe el «área de meseta»), y la curtosis (de −0,3 a 3.997).

3.3 El panel 1: `spp.panel1.<métrica>.*`

Nombre completo del hecho, en su propia línea:

`spp.panel1.<métrica>.{real, median, p1, p5, p10, p25, p50, p75, p90, p95, p99}`

con `<métrica>` = `net_profit`, `profit_factor`, `retorno_drawdown`, `max_drawdown`, `sharpe`, `sortino`.

- *Qué mide:* la distribución de cada métrica entre todos los vecinos, y dónde cae el backtest real.
- *De qué rejilla:* **sólo la del `build`** («Solo IS»), con el original **fuera** (`one.py:61`). La ventana tiene tres periodos (Solo IS, Solo OOS1, combinado); el extractor de hechos se queda con el periodo por defecto, que es el primero (`pipeline/autopilot/facts.py`, función `flatten`). Lo he comprobado: los 66 hechos coinciden con los percentiles de la rejilla `SPP_IS` (diferencia relativa máxima 5×10^{-6}) y no con los de `SPP_OOS`.
- *Cómo se calcula:* `.real` es la métrica de la permutación −1 (el backtest original en el `build`); `.median` y `.p50` son el mismo número; los percentiles, como en 2.3 (`core/study/blocks.py:113-115`).

Beneficio neto (USD, 8 años de `build`)

	real	p5	mediana	p95
C	13.239	−4.385	9.642	20.415
E	22.508	3.753	17.443	35.299
F	2.944	−27.813	−7.012	16.910
G	53.725	15.163	36.815	57.231

Cómo leerlo. C: su backtest da 13.239, pero el vecino típico da 9.642 (1.205 USD al año); uno de cada veinte vecinos pierde más de 4.385. F: el backtest gana 2.944, y el vecino típico **pierde 7.012**; el real está en el 77 % de su rejilla. G: el vecino típico da 36.815 y hasta el peor 5 % gana 15.163; el real, 53.725, es un 46 % más que la mediana. Rango de la mediana en las diez: −2.971 / 16.228 / 31.433.

Profit factor

	real	p5	mediana	p95
C	1,47	0,94	1,57	2,21
E	1,59	1,06	1,47	2,83
F	1,04	0,72	0,93	1,17
G	1,80	1,10	1,58	2,47

Cómo leerlo. El pivote es 1. C es curiosa: su PF real (1,47) está **por debajo** de la mediana de su vecindario (1,57) aunque su beneficio esté por encima; hay vecinos que operan menos y con mejor PF. Un p5 por encima de 1 (E, G) quiere decir que 19 de cada 20 vecinos ganan. Rango del p5: 0,77 / 1,005 / 1,08; de la mediana: 1,02 / 1,37 / 1,58.

Retorno/Drawdown

	real	p5	mediana	p95
C	2,47	−0,37	2,92	7,69
E	3,43	0,40	2,70	7,65
F	0,30	−0,81	−0,35	1,43
G	8,65	1,02	4,35	8,95

Cómo leerlo. Es la métrica del veredicto y la que SQX optimizó. Crece con la longitud de la ventana, así que este número de 8 años no se compara con uno de 4. Rango de la mediana: 0,08 / 2,23 / 3,75.

Drawdown máximo (USD) — aquí **alto es malo**.

	real	p5	mediana	p95
C	5.370	1.690	3.348	15.960
E	6.562	2.233	6.658	15.600
F	9.815	8.390	19.436	46.469
G	6.212	4.266	8.598	21.022

Cómo leerlo. La cola que importa es la **alta**: el p95 es «la caída que habrías tenido con mala mano en los parámetros». Para C, 15.960 frente a los 5.370 del backtest: tres veces más. F: su backtest tiene una caída de 9.815, pero el vecino típico cae 19.436; el generador eligió, entre sus vecinos, uno de los de menor caída (rango 9 %). Es la misma lección que el «dimensiona con el doble del drawdown del backtest» del Monte Carlo, vista por otro lado. Rango del cociente p95/real en las diez: de 2,4 a 5,1.

Sharpe y Sortino (anualizados, como los da SQX)

	Sharpe real	Sharpe p5	Sharpe mediana	Sharpe p95
C	0,52	−0,17	0,51	0,86
E	0,65	0,10	0,50	0,91
F	0,08	−0,61	−0,19	0,36
G	1,01	0,24	0,72	1,08

Cómo leerlo. No crecen con la ventana: son los que se pueden comparar entre `build` y `oos1`. Rango de la mediana del Sharpe: −0,04 / 0,49 / 0,65. El Sortino sigue el mismo dibujo (mediana de C 1,18; E 0,85; F −0,25; G 1,28).

Lo que hay en la ventana y no en los hechos: los mismos seis histogramas para «Solo OOS1» y para «IS + OOS1 combinado», y la banda de ±30 %. Lo de `oos1`, calculado por mí desde la rejilla:

beneficio neto, `oos1` (4 años)	real	p5	mediana	p95
C	4.690	−1.533	6.947	17.694
E	19.670	282	13.894	32.593
F	5.626	−10.577	12.637	29.264
G	22.721	3.377	19.969	35.291

F fuera de muestra es otra estrategia: su vecino típico gana 12.637 en 4 años, cuando en el `build` perdía 7.012 en 8.

3.4 El panel 2: `spp.panel2.<métrica>.*`

`spp.panel2.<métrica>.{median, p1, p5, p10, p25, p50, p75, p90, p95, p99}`

con `<métrica>` = `profit_factor`, `sharpe`, `sortino`, `r_edge_ratio`. No hay `.real`.

- *Qué mide en la ventana:* las rejillas del `build` y de `oos1` superpuestas, sólo en métricas que no crecen con la ventana, para comparar su forma.
- *Qué es el hecho, exactamente* (`contract.py:125`): los percentiles de **las dos rejillas juntas en un solo montón** (los valores IS y los OOS1 concatenados, sin el original). Comprobado: coincide con esa mezcla al octavo decimal.
- *Ejemplo real, C, Sharpe:* mediana de la mezcla 0,49; p5 −0,20; p95 0,94. La mediana IS era 0,51 y la de `oos1` es 0,48.
- *Cómo leerlo:* **con cuidado, o no leerlo.** La mediana de una mezcla de dos ventanas no es la mediana de ninguna de las dos ni la de un backtest conjunto. Cuando IS y OOS1 se parecen (C, G) coincide con ambas; cuando no (F: Sharpe mediano −0,19 en IS, +0,45 en `oos1`, mezcla +0,06), el número del hecho describe algo que no existe. La comparación que la ventana dibuja —dos curvas, cada una con su porcentaje por encima del pivote— no llega a los hechos. Rango de la mediana del Sharpe mezclado: 0,12 / 0,49 / 0,66.
- *R/Edge ratio:* cuánto se mueve el precio a favor de la entrada frente a en contra, en pips; por encima de 1 hay ventaja de entrada. Mediana mezclada: C 1,36; E 1,13; F 0,92; G 1,19.

3.5 El diseño de variantes (no es un hecho, pero es la salida que usa el paso 16.5)

`design_brief_<estrategia>.json`, uno por estrategia. Para cada parámetro vivo: el centro de su meseta, los niveles en que se fabricarán las variantes (hasta 9, cubriendo al menos ±30 % del original), el valor original, el mejor nivel en el `build`, la anchura de la meseta y la marca `spike` (anchura 1). Para cada congelado, su valor y el motivo. Y el veredicto de ruido: si fuera RUIDO, el paso 16.5 no fabricaría.

Cómo se calcula la meseta de un parámetro (`model/profile.py`): para cada nivel del parámetro, la **mediana** del Ret/DD de todas las combinaciones que usaron ese nivel (sin el original); la meseta es el tramo **contiguo** de niveles alrededor del mejor que no baja más de la mitad del recorrido entre el mejor y el peor nivel.

En las diez: todas `proceed`. Parámetros con `spike`: C 3 de 14; E 1 de 6; F 2 de 7; G 3 de 13.

3.6 Las «métricas para criterios»: `spp.verdict_csv.*` (en disco hoy; no en los hechos de la corrida)

Las escribe `report.py` en `verdict.csv` desde el 2026-10-04 por la noche. El propio módulo las titula «sin umbral todavía» y dice que no juzgan nada. Son 36 números y `spp.verdict_csv.drop` (0 en las diez). Dos familias.

A. El original frente a sus permutaciones — 24 números: para cada una de las seis métricas del panel 1 y cada ventana (IS, OOS1):

`spp.verdict_csv.distancia_pct_<ventana>_<métrica>` y `spp.verdict_csv.distancia_iqr_<ventana>_<métrica>`

- *Cómo se calculan* (`model/decay.py:14`):

```
distancia_pct = 100 × (real − mediana) / |mediana|        sólo si |mediana| ≥ 0,25 × IQR
distancia_IQR = (real − mediana) / IQR                     IQR = p75 − p25
```

- *Ejemplo real, C, beneficio neto en el* `build`, a mano con los hechos del panel 1:

```
real − mediana = 13.238,53 − 9.642,42 = 3.596,11
distancia_pct  = 100 × 3.596,11 / 9.642,42 = +37,29 %          guardado: 37,29  ✓
IQR            = 15.467,80 − 4.467,38 = 11.000,42
distancia_IQR  = 3.596,11 / 11.000,42 = +0,327                  guardado: 0,327  ✓
```

- *Cómo leerlo:* la primera es el criterio heredado de SQX («el real dentro de ± X % de la mediana»). Es engañosa: F tiene +142 % en el `build` porque su mediana es negativa y pequeña; y queda sin dato cuando la mediana roza el cero (`13.3.390` en `oos1`). La segunda es la buena: la distancia en **anchuras del vecindario**, que es la medida de «pico» del apartado 2.8. En las diez, en beneficio neto IS: de −0,38 a +0,93. La ventana enseña al lado el rango en que cae el 90 % de los propios vecinos (para C, de −1,28 a +0,98): un real fuera de ese rango sería un extremo. **Ninguna de las diez lo está.**

`distancia_iqr_…`	IS neto	OOS1 neto	IS Ret/DD	OOS1 Ret/DD
C	+0,33	−0,26	−0,11	−0,13
E	+0,36	+0,40	+0,28	+0,62
F	+0,59	−0,39	+0,88	−0,16
G	+0,93	+0,20	+1,32	+0,20

B. IS contra OOS1 como dos muestras — 12 números: para Sharpe, Sortino, profit factor y R/Edge:

`spp.verdict_csv.decaimiento_<métrica>`, `spp.verdict_csv.a_<métrica>`, `spp.verdict_csv.wasserstein_<métrica>`

- **`decaimiento`** = mediana(OOS1) / mediana(IS). Es un cociente de medianas porque las dos rejillas no comparten combinaciones: no hay «la misma variante en las dos ventanas». 1 = no decae. Sólo se publica si la mediana IS supera un suelo (Sharpe y Sortino 0,1; PF y R/Edge 1), porque con una mediana IS cerca de cero el cociente estalla o cambia de signo. *C, Sharpe:* 0,48 / 0,51 = 0,94 ✓ (guardado 0,941). *F:* sin dato (su mediana IS es −0,19).
- **`a`** = A de Vargha–Delaney: la probabilidad de que un vecino `oos1` tomado al azar supere a un vecino IS tomado al azar (más la mitad de los empates). 0,5 = distribuciones iguales; por debajo, decae. *C, Sharpe:* 0,499. *`13.3.390`:* 0,105 (su `oos1` es casi siempre peor que su IS). *F:* 0,826 (al revés).
- **`wasserstein`** = cuántos IQR de la rejilla IS hay que desplazar una distribución para convertirla en la otra. 0 = iguales. Es bilateral: también crece si `oos1` es **mejor**. *C, Sharpe:* 0,11. *`13.3.390`:* 1,69. *F:* 1,32.

Cómo leerlas, y el límite. Las tres dicen lo mismo de tres maneras: si el vecindario entero se movió entre el `build` y `oos1`. La ficha `knowhow/research/spp-decay-metrics-zero-edge-null.md` lo deja claro: con un `oos1` de 4 años, una familia sin ventaja cae en Sharpe 0 ± 0,5, y ese intervalo cubre «no decae» para cualquier mediana IS por debajo de unos 0,8. **Son diagnósticos, no criterios**, hasta que se corra un SPP sobre una población sin ventaja o sobre un `oos1` más largo.

3.7 Lo que trae el export de SQX y el estudio no usa

La tabla resumen de SQX (`runs.parquet`) trae, por estrategia y ventana: permutaciones, rentables, perdedoras, **porcentaje de rentables**, **beneficio medio**, **desviación típica** y beneficio máximo. Es decir, la media y la desviación que el Council echaba de menos **ya están en el export**; no llegan a los hechos porque el extractor no lee esa tabla. Ejemplo, C en el `build`: 90,09 % rentables, beneficio medio 9.274,72, desviación 8.164,55 (coinciden con lo que calculo desde la rejilla).

Un detalle: SQX cuenta como «rentable» la permutación que no opera (beneficio 0). En `13.3.390` da 97,45 %; con beneficio estrictamente mayor que cero son 94,88 % (la diferencia son las 333 que no operan).

---

4. El criterio cargado y por qué

4.1 Tal como está hoy en disco

`pipeline/autopilot/criteria.yaml`, entrada `"16"`: corte sobre «SPP OOS», segmento `oos1`, **`rules: []`**. El modo de desarrollo (`dev.on: true`) sólo sortea en el paso 8. No hay línea `frozen`.

regla	Pasa	Limbo	Falla
(ninguna)	todas	—	—

La razón: la del apartado 2.9. El Council lo quiso de sólo descarte, con dos condiciones que no estaban expuestas como hecho; no hay tasa de falsos positivos medida para ninguna regla de este paso; y sobrevivir aquí no es evidencia, porque relee las mismas operaciones.

4.2 Lo que haría falta para la regla del Council, y qué se puede sacar ya de los percentiles

El encargo lo listaba así: porcentaje de vecinos rentables, media y desviación de la rejilla, y una marca de pico. Estado real de cada una:

lo que falta	¿existe hoy como hecho?
% de vecinos rentables	**Sí**, `spp.verdict.area_de_meseta` (Ret/DD > 0, `build`)
media de la rejilla	No. Está en el export de SQX (`avg_profit`)
desviación de la rejilla	No. Está en el export de SQX (`stdev`)
media − 1 desv	No. Aproximable con p10 y p25
marca de pico	No, y no está definida
cualquiera de ellas sobre `oos1`	No

Aproximar «media − 1 desviación» con los percentiles. En una distribución normal, media − 1 desv es el percentil 15,87. No hay p16, pero está entre p10 y p25. Tres maneras, de más simple a más fina:

```
(a) p10 directamente              algo más estricta: p10 queda por debajo de p16
(b) p16 ≈ p10 + 0,464 × (p25 − p10)
        0,464 = (−1 − z₁₀) / (z₂₅ − z₁₀) = (−1 + 1,2816) / (−0,6745 + 1,2816)
(c) mediana − IQR / 1,349         en una normal, IQR = 1,349 desviaciones
```

Sobre el beneficio neto del `build`, con los hechos de `spp.panel1.net_profit.*`:

	exacta: media − desv	**(a) p10**	**(b) p16 aprox.**	**(c) mediana − IQR/1,349**
C	+1.110	+69	+2.109	+1.488
E	+8.327	+6.299	+8.483	+7.133
F	−19.552	−22.379	−18.905	−19.618
G	+23.592	+19.716	+23.273	+23.278

Cuenta de C por la vía (b): 68,85 + 0,464 × (4.467,38 − 68,85) = 68,85 + 2.040 = 2.109. La exacta sale de la rejilla: media 9.274,72 − desviación 8.164,55 = 1.110,17. La aproximación acierta el signo en las cuatro y el tamaño dentro de unos 1.000 USD; en C se desvía más porque su distribución tiene una cola larga hacia abajo (su p1 es −17.910).

Aproximar el % de rentables con «dónde cae el cero entre los percentiles». Se busca entre qué dos percentiles queda el cero y se interpola:

```
C: el cero está entre p5 (−4.385,16) y p10 (+68,85)
   rango del cero ≈ 5 + 5 × 4.385,16 / (68,85 + 4.385,16) = 5 + 4,92 = 9,92
   rentables ≈ 100 − 9,92 = 90,1 %                                exacto: 90,09 %
F: entre p50 (−7.012,04) y p75 (+2.117,86)
   rango ≈ 50 + 25 × 7.012,04 / 9.129,90 = 69,2  →  rentables ≈ 30,8 %    exacto: 29,8 %
E: entre p1 (−2.465,54) y p5 (+3.752,75)
   rango ≈ 1 + 4 × 2.465,54 / 6.218,29 = 2,59   →  rentables ≈ 97,4 %     exacto: 98,0 %
```

Aproximar el rango del real (apartado 2.4): interpolando el real entre sus dos percentiles vecinos. G: 93,7 frente a 93,9 exacto.

No usar media − desviación sobre el Retorno/Drawdown. Una sola permutación de 6 operaciones con Ret/DD = 554 hincha la desviación de `13.3.390` a 44,9, y «media − desv» da −41,6 para una rejilla con el 97 % de vecinos rentables. Sobre Ret/DD la cuenta exacta falla a seis de diez; la del percentil, a dos. La regla de Kaufman se formula sobre **beneficio**; ahí sí funciona.

4.3 Cuántas fallaría cada regla posible, sobre la población real

Ninguna está cargada. Las cuentas son mías, sobre las diez. Las del `build` usan hechos que existen; las de `oos1`, la rejilla en bruto.

Sobre la rejilla del `build` (expresable hoy con hechos)

regla candidata de descarte	fallan	quiénes
media − desv del neto < 0 (exacta)	3	F, `16.5.352`, `19.2.363`
p16 aproximado del neto < 0	3	las mismas
mediana − IQR/1,349 del neto < 0	3	las mismas
`spp.panel1.net_profit.p10` < 0	3	las mismas
`spp.panel1.net_profit.p5` < 0	4	las mismas y C
`spp.verdict.area_de_meseta` < 0,70 (Kaufman)	2	F, `19.2.363`
`area_de_meseta` < 0,50	1	F
media del neto < 0	2	F, `19.2.363`
`spp.verdict.pass` = 0	0	—
pico: rango del real ≥ 95 %	0	— (máx. 94, G; umbral sin calibrar, puesto justo por encima de lo observado: que marque 0 de 10 no valida la regla)

Sobre la rejilla de `oos1` (no expuesto; calculado)

regla candidata de descarte	fallan	quiénes
media − desv del neto < 0 (exacta)	5	`13.3.390`, F, `16.5.352`, `18.5.355`, `19.2.363`
p16 aproximado del neto < 0	4	las mismas menos `18.5.355`
p10 del neto < 0	6	las cinco y C
p5 del neto < 0	8	todas menos E y G
% rentables < 70 %	2	`13.3.390`, `16.5.352`
media del neto < 0	1	`13.3.390`

Lo que se ve:

- **La regla del Council, «media − 1 desv < 0», falla 3 de 10 en el `build` y 5 de 10 en `oos1`.** La unión de las dos, 5 de 10. Con `oos1` de 4 años la rejilla es más ancha en proporción y la regla aprieta más.
- **Las aproximaciones aciertan en 3 de 3 en el `build` y en 4 de 5 en `oos1`.** La que se escapa, `18.5.355`, está en el filo: exacta −492, aproximada +563.
- **El 70 % de Kaufman es más blando que «media − desv»:** 2 fallos frente a 3.
- **Los dos conjuntos no son los mismos en las dos ventanas.** F falla por el `build` y no por `oos1` en la regla del 70 %; `13.3.390` al revés. Elegir ventana es elegir a quién se descarta.
- **`pass` no falla a nadie.**

4.4 Cómo sería el resultado ideal

En números, para una estrategia de este tipo (unas 20 operaciones al año, 8 años de `build`):

- **No existe un resultado ideal certificable en este paso.** Como referencia, no como umbral (ninguna población la ha calibrado): en `oos1`, rango del real entre 40 y 80 %; en el `build` se espera más alto (mediana medida 66), y lo anómalo ahí sería 50. Distancia a la mediana: por debajo de 1 IQR es lo tranquilo, por encima de 1,5 candidato a pico (G, con +1,32, queda entre ambos).

- **La cola baja en positivo:** p10 del beneficio neto mayor que cero (sólo equivale a media − 1 desv > 0 en una normal, y aun ahí es algo más estricto: el p10 de una normal es μ − 1,28σ; con colas largas difieren hasta un factor 2, como en C, y que coincidan en las diez es una observación de esta población); mejor aún, p5 mayor que cero. G lo cumple en las dos ventanas; E también.
- **El vecindario gana:** más del 90 % de vecinos rentables. Sabiendo que esto, solo, lo consigue casi cualquier familia elegida por ganar en el `build`.
- **Cociente p95/real del drawdown:** en las diez va de 2,4 a 5,1 y G, «la que mejor sale», tiene 3,4 (21.022 / 6.212); no hay valor ideal medido.
- **La misma imagen en `oos1`:** un A de Vargha–Delaney cerca de 0,5 también lo da una familia sin ventaja, y tras elegir por IS lo normal es A < 0,5; no es un criterio. Mediana `oos1` por año parecida a la del `build`. G: 36.815 / 8 = 4.602 USD al año en el `build` y 19.969 / 4 = 4.992 en `oos1`.

El techo honrado. Ni la mejor imagen posible prueba ventaja: los vecinos comparten histórico (2.7) y, en una corrida normal, `oos1` ya se usó para elegir (2.9). G es la que mejor sale aquí y en el paso 10 su t conjunta en otros mercados era negativa, como la de las diez. Este paso puede decir «no hay señales de pico»; no puede decir «es robusta».

4.5 Si mueves un umbral

Como no hay regla, «mover» es «poner». Coste en estrategias sobre las diez, y lo que se gana:

si pones…	fallan	qué ganas	qué no sabes
nada (hoy)	0	nada; el paso sólo anota	—
`net_profit.p10 ≥ 0` (`build`) como Falla	3	quitas las de cola baja negativa: la regla del Council, aproximada	tasa de falsos positivos: no medida
lo mismo como Limbo	0 (3 en Limbo)	marcas sin borrar	—
`area_de_meseta ≥ 0,70` como Falla	2	el 70 % de Kaufman	con ρ = 0,9 supuesto (no medido), el modelo de 2.7 da un 86 %; con ρ menor baja y con vecinos no gaussianos puede cambiar. Es una ilustración, no una tasa de falsos positivos
`net_profit.p5 ≥ 0` como Falla	4	más estricta	se lleva a C, cuyo vecindario gana en el 90 %
regla sobre `oos1` (hay que exponerla)	5 con media − desv	usa la ventana que el generador no vio	en corrida normal, `oos1` ya está usada

Más estricto quita más de las que pierden con parámetros vecinos, y también más de las que simplemente tienen una rejilla ancha (muchos parámetros vivos, pocas operaciones). **Más laxo** deja pasar todo, que es lo de hoy. Lo que ninguna opción te da es una probabilidad: sin una población sin ventaja pasada por este paso, no hay forma de saber cuántas de las tres (o cinco) descartadas eran malas de verdad.

Una referencia cruzada, sin valor de prueba: F, que aquí falla todas las candidatas del `build`, era también la peor de las diez por t conjunta en el paso 10; E, que las pasa todas, era la mejor.

---

5. Lo que no cuadra o no está medido

5.1 «Área de meseta» no es lo que dice su glosario

El hecho es la fracción de combinaciones con Ret/DD > 0 (`core/surface/plateau.py:24` , cuyo comentario dice «fracción de la rejilla que gana dinero»). El glosario de la ventana (`one.py:35`) dice «parte de la rejilla cuyo resultado queda cerca del mejor nivel: cerca de 1 es una zona ancha y estable». Eso describe `above_half_max` , que vale entre 0,0001 y 0,044 en las diez y no se publica. Con el nombre actual, un 0,96 se lee como «el 96 % de la rejilla está cerca del máximo», y lo que quiere decir es «el 96 % gana algo».

5.2 El paso corta «SPP OOS» y todos sus hechos son del `build`

El estudio se lanza sobre `SPP_IS` (sólo hay carpeta de resultados para ese databank). El veredicto se calcula sobre esa rejilla, y del panel 1 los hechos sólo recogen «Solo IS». Cualquier regla escrita hoy con `spp.panel1.*` o `spp.verdict.*` juzgaría el `build` , aunque la entrada del criterio diga `segment: oos1` . No es necesariamente malo —la regla de Kaufman es de dentro de muestra— pero hay que saberlo.

5.3 El veredicto propio no tiene suelo de operaciones y lo deciden permutaciones que casi no operan

Apartado 2.5. Siete de los diez máximos tienen menos de 30 operaciones. El filtro de centinelas (`dedupe.drop_sentinels`) sólo quita filas con `RExpectancy` fuera de rango; no mira el número de operaciones. La anotación del código dice «sin suelo de operaciones (dueño, 2026-10-04)».

5.4 El veredicto propio aprueba picos y suspende mesetas

Apartado 2.5, ejemplo 3×3. Es coherente con su pregunta («¿merece variantes?») e incoherente con su nombre en los hechos (`pass`) y con el texto de la ventana («aprueba / no aprueba»). El README del estudio dice que se lee junto con el área y el pico; los hechos lo publican suelto.

5.5 `n_eff` se llama efectivo y sólo quita duplicados exactos

Apartado 2.6. El comentario de `dedupe.n_eff` habla de «autocorrelación de la rejilla», pero la función cuenta filas distintas. La correlación entre vecinos no entra en ninguna cuenta.

5.6 El panel 2 de los hechos mezcla dos ventanas en un solo percentil

Apartado 3.4. En la ventana son dos curvas; en los hechos, una. En F la mezcla (+0,06 de Sharpe) no se parece a ninguna de las dos (−0,19 y +0,45).

5.7 Un hecho duplicado

`spp.verdict.tuplas_distintas_n_eff` = `spp.summary.n_eff` , en las diez.

5.8 Los hechos de la corrida no tienen las métricas para criterios

114 claves en el `hechos.parquet` del 2026-10-04 16:35; 151 si se extrajeran hoy de la misma carpeta, que se reescribió a las 22:02. Las 37 que faltan son las `spp.verdict_csv.*` .

5.9 El tope de 15.000 no se alcanza

Entre 10.239 y 14.406 filas por estrategia. No sé si SQX descarta repeticiones, combinaciones inválidas o se queda sin combinaciones en los parámetros enteros. **No medido.**

5.10 «6 combinaciones en común de 11.600»

Es la medida del README sobre otra estrategia (`17.9.39`, XAUUSD). En las dos que he comprobado aquí hay **cero**. La conclusión es la misma —dos SPP no se emparejan— y más fuerte.

5.11 Los desplazamientos no se mueven ±35 %

Van de 0 a 6 partiendo de 1 (apartado 1.2). El estudio lo sabe para el diseño de variantes (los deja fijos), pero en la rejilla SPP sí se mueven, y un desplazamiento de 6 barras cambia la regla mucho más que un 35 % de un periodo. Parte de la anchura de cada rejilla viene de ahí. Cuánta, **no medido**.

5.12 SQX cuenta como rentable la permutación que no opera

Apartado 3.7. Afecta a su `profitable_pct` y, si un día se reactiva, a su `ProfitOptPct 70`.

5.13 Lo que no está medido

- La tasa de falsos positivos de **cualquier** regla de este paso (paquete de evidencia, «Unmeasured: any FPR for steps 14-19»).
- Una población sin ventaja pasada por el SPP.
- La correlación entre vecinos, y con ella un n efectivo de verdad.
- Si el porcentaje de rentables o la mediana del `build` predicen los de `oos1` en una población grande. Aquí, con diez: correlación de rangos +0,59 (de −0,07 a +0,89) en el porcentaje; +0,69 en la mediana del Sharpe.
- Qué pasa con este paso sobre estrategias que **sí** pasan el 8. Ninguna corrida lo tiene.
- Una definición operativa de «solución pico» con todos los parámetros moviéndose a la vez.

5.14 Dónde este documento corrige al informe anterior y al encargo

- **«`area_de_meseta`: qué parte de la rejilla queda cerca del mejor nivel»** (informe anterior). Es el porcentaje de vecinos con Ret/DD > 0.
- **«`spp.panel2.*`: lo mismo comparando IS con oos1»** (informe anterior). Es la mezcla de las dos en un solo montón.
- **«unas 11.500 combinaciones distintas de 15.000»** (informe anterior). Son 11.470 de mediana sobre entre 10.239 y 14.406 filas; a 15.000 no llega ninguna.
- **«supone N independientes […] así que sobrestima»** (informe anterior). La correlación entre vecinos hace el listón más estricto, no más laxo; lo que sobrestima es la cola del Ret/DD (2.6).
- **«el % de vecinos rentables no está expuesto como hecho»** (encargo de este documento). Sí lo está, para el `build`, como `area_de_meseta`. Lo que no está expuesto es la media, la desviación, una marca de pico y todo lo de `oos1`; el comentario de `criteria.yaml`, que sólo nombra el pico y la media menos una desviación, es exacto.
- **Los percentiles del panel 1 «sobre IS / OOS»** (encargo). Son sólo IS.

- Lo demás del informe anterior coincide con mis cuentas: PF real de mediana 1,48 frente a 1,37 de la rejilla; p5 del PF de mediana 1,005; área entre 0,23 y 1,00 con mediana 0,96; `score` entre 1,3 y 19,3; `pass` = 1 en las diez.

---

6. Cierre

6.1 Las reglas, en una tabla

regla	hecho	Pasa	Limbo	Falla
—	—	todas	—	—

6.2 El resultado sobre la población real

Calculado en memoria con `pipeline.autopilot.criteria.outcomes` sobre los 1.140 hechos de la corrida y las reglas de hoy:

	Pasa	Limbo	Falla
diez estrategias	**10**	0	0

Y lo que dirían las lecturas de este documento, una al lado de otra:

	% rentables IS / OOS1	media − desv neto IS / OOS1	rango Ret/DD IS / OOS1
C `14.3.479`	89 / 88	+1.110 / +1.360	44 / 43
E `3.11.426`	98 / 95	+8.327 / +4.736	64 / 76
F `15.9.352`	23 / 75	−19.552 / −1.223	79 / 41
G `4.13.338`	99,6 / 98	+23.592 / +9.889	94 / 61

6.3 Las decisiones que te quedan

Cada una con su coste en estrategias (sobre las diez) junto a lo que gana.

1. **¿Dejar el paso sin regla?** Coste: 0. Ganancia: 0. Es lo que hay y lo que el Council aceptó mientras no existieran los hechos. El paso sigue sirviendo para lo que sí hace bien: escribir el diseño de variantes.

2. **¿Poner ya la regla de descarte del Council, aproximada con un hecho que existe?** `{fact: spp.panel1.net_profit.p10, pass: ">= 0"}`. Coste: **3 de 10** (F, `16.5.352`, `19.2.363`), las mismas que la cuenta exacta. Ganancia: quitar las familias cuya cola baja pierde en el `build`. Lo que no sabes: su tasa de falsos positivos. Alternativa sin coste: la misma línea con `else: limbo`, que marca sin borrar.

3. **¿El 70 % de Kaufman?** `{fact: spp.verdict.area_de_meseta, pass: ">= 0.7"}`. Coste: **2 de 10** (F, `19.2.363`). Ganancia: poca; es más blanda que la anterior y el modelo ilustrativo del apartado 2.7 da un 86 % de familias elegidas por ganar que la pasan, con ρ = 0,9 supuesto (no medido); con ρ menor baja y con vecinos no gaussianos puede cambiar. No es una tasa de falsos positivos. El Council la admitía como convención de Pasa, nunca como única evidencia.

4. **¿Juzgar el `build` o `oos1`?** Hoy sólo se puede el `build`. Sobre `oos1`, «media − desv < 0» costaría **5 de 10**, y no las mismas. A favor de `oos1`: es la ventana que el generador no vio. En contra: en una corrida normal el paso 8 ya eligió mirándola, y con 4 años la rejilla es más ruidosa. Requiere exponer los hechos de «Solo OOS1».

5. **¿Exponer lo que falta?** Media, desviación y media − desv del beneficio (ya vienen en el export de SQX), el rango del real, la distancia del real en IQR (ya en `verdict.csv`), y los percentiles de `oos1`. Coste en estrategias: 0. Es trabajo de código, no una decisión estadística. Sin esto, las decisiones 2 a 4 se toman con aproximaciones.

6. **¿Qué hacer con `spp.verdict.pass`?** Tres salidas: (a) dejarlo y no leerlo nunca como criterio (coste 0); (b) ponerle un suelo de operaciones —con 30, C pasaría a RUIDO y no se le fabricarían variantes: **1 de 10**—; (c) calcularlo sobre una métrica sin cola de cociente (Sharpe o beneficio). En cualquier caso, renombrarlo: hoy la ventana dice «aprueba» de algo que aprueba picos.

7. **¿Renombrar `area_de_meseta`?** Coste 0. Evita que se lea al revés, como ya le pasó al informe anterior.

8. **¿Definir la marca de pico?** La candidata más simple con lo que existe: real por encima del p95 de su rejilla **y** a más de 1,5 IQR de la mediana. Coste hoy: **0 de 10**. No se puede calibrar hasta ver picos de verdad. Umbral sin calibrar: está puesto justo por encima de lo observado (máximo 94 % y 1,32 IQR, los dos de G, que queda a un paso). Que marque 0 de 10 no valida la regla.

9. **¿Correr la prueba de vacuidad?** Pasar una población sin ventaja («monos seleccionados») por los pasos 15 y 16 y contar cuántos pasan cada regla candidata. Coste: una corrida de SPP (26 minutos por cada diez estrategias). Ganancia: la única cifra que convierte cualquiera de las reglas de arriba en algo más que una convención. Si más del 90 % de los nulos pasa una regla, esa regla es vacía.

6.4 El límite que no cambia

El SPP contesta una pregunta útil y estrecha: **cuánto del resultado depende de haber acertado con los parámetros**. La contesta bien con tres números —la mediana del vecindario, su cola baja y el puesto del real— y mal con el que hoy lleva el nombre de veredicto. No contesta si la idea tiene ventaja: todos los vecinos viven en el mismo histórico, y una meseta dentro de muestra es compatible con que no quede nada fuera. Por eso el sitio natural de este paso es el que le dio el Council: **descartar lo claramente frágil, anotar lo demás, y no contar la supervivencia como evidencia.**